SENG 319: Software Design Patterns

Group Project 1: Detection and Refactoring of Anti-Patterns and Code Smells
Group Name: Tozaring | Target Codebase: CheckStyle 14.1.0

Part I: Anti-Patterns

1. Anti-Pattern Types (Design Smells) Detected by PTIDEJ

PTIDEJ reported the following design smell types on CheckStyle (124 instances in total):

Anti-Pattern / Design Smell Type	Instances	PTIDEJ Output File
`RefusedParentBequest`	40	`RefusedParentBequest.ini`
`ComplexClass`	31	`ComplexClass.ini`
`LongParameterList`	19	`LongParameterList.ini`
`LazyClass`	13	`LazyClass.ini`
`Blob`	7	`Blob.ini`
`ClassDataShouldBePrivate`	7	`ClassDataShouldBePrivate.ini`
`AntiSingleton`	3	`AntiSingleton.ini`
`BaseClassShouldBeAbstract`	2	`BaseClassShouldBeAbstract.ini`
`SpeculativeGenerality`	2	`SpeculativeGenerality.ini`
`BaseClassKnowsDerivedClass`	0	`BaseClassKnowsDerivedClass.txt`
`FunctionalDecomposition`	0	`FunctionalDecomposition.txt`
`LargeClass`	0	`LargeClass.txt`
`ManyFieldAttributesButNotComplex`	0	`ManyFieldAttributesButNotComplex.ini`
`MessageChains`	0	`MessageChains.ini`
`SwissArmyKnife`	0	`SwissArmyKnife.ini`
`TraditionBreaker`	0	`TraditionBreaker.ini`
Total Design Smells (Anti-Patterns)	**124**	**16 files**

2. Anti-Pattern with the Most Impact on CheckStyle Quality

In our opinion the design smell with the most impact is `ClassDataShouldBePrivate`. A class that exposes non-private data lets any other class read and change its state, so the class can no longer protect itself and every user of the class depends on its internals. `LongParameterList` is a close second: methods with long argument lists are hard to read, easy to call with arguments in the wrong order, and hard to change.

The other two smells we looked at are also serious:

- `Blob`: one large class that holds all the logic while the other classes only hold data. It wastes lines of code, is hard to maintain and wastes memory, because the same work gets repeated inside the large class.
- `AntiSingleton`: a class that exposes mutable static (global) state. Any thread can change it at any time, which can cause race conditions and hard-to-reproduce bugs.

We chose to refactor `Blob` and `AntiSingleton` as they are severe issues where a small, targeted fix gives a big gain in safety, performance and maintainability relative to the effort.

3. Refactoring Two Anti-Pattern Instances

Instance 1: AntiSingleton in JavaLanguageParser

a. Reason for choosing the Anti-Pattern instance:

- **Severity and Risk:** A mutable static field acts as uncontrolled global state. In multi-threaded execution, concurrent reads and writes can trigger data races, lost increments, or premature resets, corrupting runtime decisions.
- **High Impact Location:** `JavaLanguageParser` is instantiated on the hot path for every single source file parsed by CheckStyle, so resolving concurrency and memory leaks here protects the entire analysis pipeline.
- **Effort vs. Gain:** It can be refactored cleanly in-place with a high return on investment, eliminating global state while keeping the DFA clearing behavior fully intact.

b. Location within the CheckStyle code base [original code]:

`com.puppycrawl.tools.checkstyle.grammar.java.JavaLanguageParser` (defined in `JavaLanguageParser.g4`):

```
    static int fileCounter = 0; // PROBLEM: package-private mutable static field (unprotected gl
obal state)

    public JavaLanguageParser(TokenStream input, int clearDfaLimit) {
        super(input);
        _interp = new ParserATNSimulator(this, _ATN , _decisionToDFA, _sharedContextCache);
        fileCounter++; // PROBLEM: non-atomic increment susceptible to data races across threads
        if (fileCounter > clearDfaLimit) { // PROBLEM: unsynchronized check and reset
            _interp.clearDFA();
            fileCounter = 0; // PROBLEM: concurrent threads can overwrite or double-reset
        }
    }
```

Why this is a problem:

- `fileCounter` is a mutable static field, so it is global state shared by every parser in the program, and any class in the package can change it.
- `fileCounter++`, the comparison and the reset to `0` are separate steps. When files are checked on several threads, increments can be lost or the reset can happen twice, so the DFA cache is cleared at the wrong time (memory and speed problems).

c. Steps to refactor and justification:

- **Step 1:** Create a small class `DfaClearCounter` inside `JavaLanguageParser.g4` (in the `@parser::members` block) that owns the counter as `private` fields (`limit`, `count`).
- **Step 2:** Replace `static int fileCounter` with `private static final DfaClearCounter DFA_CLEAR_COUNTER`. The reference can no longer be reassigned or reached from outside, and the number itself is hidden inside the object.
- **Step 3:** Make `tick()` `synchronized`, so incrementing, comparing and resetting happen as one step. The constructor now only asks `if (DFA_CLEAR_COUNTER.tick(clearDfaLimit))` and clears the DFA when it answers true.

d. Our implementation:

```
    private static final DfaClearCounter DFA_CLEAR_COUNTER =
            new DfaClearCounter(CLEAR_DFA_LIMIT);

    public static final class DfaClearCounter {
        private final int limit;
        private int count;

        public DfaClearCounter(int limit) {
            this.limit = limit;
        }

        public synchronized boolean tick() {
            return tick(limit);
        }

        public synchronized boolean tick(int customLimit) {
            final boolean shouldClear;
            if (++count > customLimit) {
                count = 0;
                shouldClear = true;
            }
            else {
                shouldClear = false;
            }
            return shouldClear;
        }
    }

    public JavaLanguageParser(TokenStream input, int clearDfaLimit) {
        super(input);
        _interp = new ParserATNSimulator(this, _ATN , _decisionToDFA, _sharedContextCache);
        if (DFA_CLEAR_COUNTER.tick(clearDfaLimit)) {
            _interp.clearDFA();
        }
    }
```

Why this solves the issue:

- There is no mutable static field left. The only static is a `private final` reference, and the changing number lives in a private field of the object.
- Because `tick()` is `synchronized`, the count-and-reset is atomic, so threads can no longer lose increments or reset twice. The behaviour is the same as before: the DFA is cleared once the limit is exceeded.

Result: `JavaLanguageParser` no longer has a non-private, non-final static field, which is what PTIDEJ reports as `AntiSingleton`. CheckStyle still compiles and runs.

Instance 2: Blob in AuditEventDefaultFormatter and AuditEvent

a. Reason for choosing the Anti-Pattern instance:

- **Classic Controller-DataClass Imbalance:** In this Blob instance, `AuditEvent` is reduced to an anemic data holder exposing raw attributes, while `AuditEventDefaultFormatter` assumes full responsibility for string manipulations, suffix stripping, and identifier resolution that inherently belong to the event itself.
- **Eliminating Work and Memory Waste:** Examining the original formatter reveals that `getCheckShortName(event)` was called twice for every single logged violation (once when sizing the buffer in `initStringBuilderWithOptimalBuffer` and once during message construction in `format`). This resulted in redundant `substring` heap allocations and incorrect buffer sizing whenever `moduleId` was configured.
- **High Architectural Payoff:** Moving domain logic to `AuditEvent` cures Feature Envy and restores cohesion to `AuditEventDefaultFormatter` with a compact, highly reliable refactoring.

b. Location within the CheckStyle code base [original code]:

`com.puppycrawl.tools.checkstyle.AuditEventDefaultFormatter`:

```
        final StringBuilder sb = initStringBuilderWithOptimalBuffer(event, severityLevelName); /
  / PROBLEM: first redundant call
        ...
        if (event.getModuleId() == null) {
            final String checkShortName = getCheckShortName(event); // PROBLEM: second redundant
   call to string manipulation
            sb.append(checkShortName);
        }
        else {
            sb.append(event.getModuleId());
        }
        ...
    private static StringBuilder initStringBuilderWithOptimalBuffer(AuditEvent event,
            String severityLevelName) {
        final int bufLen = LENGTH_OF_ALL_SEPARATORS + event.getFileName().length()
            + event.getMessage().length() + severityLevelName.length()
            + getCheckShortName(event).length(); // PROBLEM: Feature Envy; inspects AuditEvent i
  nternals from static helper
        return new StringBuilder(bufLen);
    }

    private static String getCheckShortName(AuditEvent event) { // PROBLEM: Domain logic isolate
  d outside AuditEvent
        final String checkFullName = event.getSourceName();
        String checkShortName = checkFullName.substring(checkFullName.lastIndexOf('.') + 1);
        if (checkShortName.endsWith(SUFFIX)) {
            final int endIndex = checkShortName.length() - SUFFIX.length();
            checkShortName = checkShortName.substring(0, endIndex);
        }
        return checkShortName;
    }
```

Why this is a problem:

- `AuditEvent` is a passive data class. All the logic that works on its data (`getSourceName()`, `getModuleId()`) sits in the formatter, which is a Blob.
- `getCheckShortName` is a static helper that uses nothing from the formatter, so the formatter class is not cohesive.
- The short name is computed twice per event, which creates extra `String` objects and wastes memory on every logged violation.

c. Steps to refactor and justification:

- **Step 1:** Move the logic to the class that owns the data. `AuditEvent` gets `getCheckShortName(String suffix)` and `getSourceIdentifier(String suffix)`, which use only its own `sourceName` and `moduleId`.
- **Step 2:** Delete the static `getCheckShortName(AuditEvent)` from the formatter and call `event.getSourceIdentifier(checkSuffix)` once in `format()`, then reuse the result for the buffer size and for the output.
- **Step 3:** Give the formatter real instance fields (`separatorLength`, `checkSuffix`) that the constructor sets and `format()` uses, so the class works on its own state.

d. Our implementation:

`AuditEvent.java` (logic moved to the data owner):

```
    public String getCheckShortName(String suffix) {
        final String checkFullName = getSourceName();
        final int lastDotIndex = checkFullName.lastIndexOf('.');
        String checkShortName = checkFullName.substring(lastDotIndex + 1);
        if (checkShortName.endsWith(suffix)) {
            final int endIndex = checkShortName.length() - suffix.length();
            checkShortName = checkShortName.substring(0, endIndex);
        }
        return checkShortName;
    }

    public String getSourceIdentifier(String suffix) {
        final String moduleId = getModuleId();
        final String identifier;
        if (moduleId == null) {
            identifier = getCheckShortName(suffix);
        }
        else {
            identifier = moduleId;
        }
        return identifier;
    }
```

`AuditEventDefaultFormatter.java`:

```
    private final int separatorLength;
    private final String checkSuffix;

    public AuditEventDefaultFormatter() {
        separatorLength = LENGTH_OF_ALL_SEPARATORS;
        checkSuffix = SUFFIX;
    }

    public String format(AuditEvent event) {
        ...
        final String sourceIdentifier = event.getSourceIdentifier(checkSuffix);
        final int bufLen = separatorLength + fileName.length()
                + message.length() + severityLevelName.length()
                + sourceIdentifier.length();
        final StringBuilder sb = new StringBuilder(bufLen);
        ...
        sb.append(": ").append(message)
            .append(" [").append(sourceIdentifier).append(']');
        return sb.toString();
    }
```

Why this solves the issue:

- The logic about an event now lives in `AuditEvent`, so it is a real class with behaviour and the formatter no longer does all the work (no more Blob).
- The static helper is gone and the formatter uses its own fields, so the class is cohesive.
- The short name is computed once per event, so there are no repeated `String` allocations and the buffer is sized with the exact length.

Result: the logic is split between the two classes in the way PTIDEJ expects, so this `Blob` instance should no longer be reported. The formatted output is identical, and CheckStyle still compiles and runs.

Part II: Code Smells

1. Code Smells Detected by PTIDEJ

PTIDEJ reported the following code smell types on CheckStyle (3,350 instances in total):

Code Smell Type	Instances	PTIDEJ Output File
ParentClassProvidesProtectedDetection	434	ParentClassProvidesProtectedDetection.ini
MethodNoParameterDetection	415	MethodNoParameterDetection.ini
NotAbstract	415	NotAbstract.ini
NoPolymorphism	401	NoPolymorphism.ini
TwoInheritanceDetection	327	TwoInheritanceDetection.ini
NotComplexClassDetection	197	NotComplexClassDetection.ini
RareOverridingDetection	163	RareOverridingDetection.ini
LongMethodClassDetection	135	LongMethodClassDetection.ini
ChildClassDetection	131	ChildClassDetection.ini
FieldPrivate	128	FieldPrivate.ini
NoInheritanceDetection	107	NoInheritanceDetection.ini
NotComplexDetection	94	NotComplexDetection.ini
ComplexClassOnlyDetection	49	ComplexClassOnlyDetection.ini
LowCohesionDetection	42	LowCohesionDetection.ini
LowCohesionOnlyDetection	42	LowCohesionOnlyDetection.ini
DataClass	36	DataClass.ini
LargeClassDetection	31	LargeClassDetection.ini
LargeClassOnlyDetection	31	LargeClassOnlyDetection.ini
ManyAttributesDetection	30	ManyAttributesDetection.ini
ClassOneMethod	25	ClassOneMethod.ini
HasChildrenDetection	22	HasChildrenDetection.ini
AbstractClass	19	AbstractClass.ini
LongParameterListClassDetection	19	LongParameterListClassDetection.ini
LongMethodDetection	16	LongMethodDetection.ini
FewMethodsDetection	13	FewMethodsDetection.ini
ControllerClassDetection	7	ControllerClassDetection.ini
FieldPublic	7	FieldPublic.ini
OneChildClassDetection	5	OneChildClassDetection.ini
ClassGlobalVariable	3	ClassGlobalVariable.ini
FunctionClassDetection	3	FunctionClassDetection.ini
NotClassGlobalVariable	3	NotClassGlobalVariable.ini
MessageChainsClassDetection	0	MessageChainsClassDetection.ini
MultipleInterface	0	MultipleInterface.ini
Total Code Smells	**3350**	**33 files**

2. Code Smell with the Most Impact on CheckStyle Quality

In our opinion the code smell with the most impact is `OneChildClass`. An abstract class with exactly one child adds a class, a file and a level of inheritance, but there is no second implementation that would ever use it. These classes are pointless, they only bloat the codebase and waste effort and memory, and they should simply be one class.

Other smells we considered:

- `FieldPublic`: public fields let any class change the data directly, which can cause bugs.
- `LongMethodClass`: a method that is much too long is hard to read, test and change, and turns maintenance into a nightmare.
- `ClassOneMethod`: a class with a single method is extra bloat.

We chose to refactor `OneChildClass` and `LongMethodClass`. `OneChildClass` is quite easy to fix and gives a good result in simplicity and maintenance, and long methods are fixed by simply breaking them up into smaller methods.

3. Refactoring Two Code Smell Instances

Instance 1: OneChildClass in the xpath package

a. Reason for choosing the Code Smell instance:

- **Unnecessary Hierarchy & File Bloat:** Both `AbstractRootNode` and `AbstractElementNode` introduce an extra abstract layer and physical files with only a single subclass (`RootNode` and `ElementNode`). They provide no polymorphic reuse or extensibility.
- **Direct Maintainability Payoff:** Applying Collapse Hierarchy consolidates the node abstractions directly into concrete classes, removing indirection and simplifying the XPath AST structure.

b. Location within the CheckStyle code base [original code]:

`com.puppycrawl.tools.checkstyle.xpath` (`AbstractRootNode.java`, `RootNode.java`, `AbstractElementNode.java`, `ElementNode.java`):

```
public abstract class AbstractRootNode extends AbstractNode { // PROBLEM: Single-child abstract
  root class
    protected AbstractRootNode() {
        super(new GenericTreeInfo(Configuration.newConfiguration()));
    }
    ... // compareOrder, getAttributeValue, getLocalPart, iterateAxis, ...
}

public class RootNode extends AbstractRootNode { // PROBLEM: Only child of AbstractRootNode
    public RootNode(DetailAST detailAst) { this.detailAst = detailAst; }
    ... // createChildren, hasChildNodes, getLineNumber, ...
}

public abstract class AbstractElementNode extends AbstractNode { // PROBLEM: Single-child abstra
  ct element class
    protected abstract AttributeNode createAttributeNode(); // PROBLEM: Hook with exactly one im
  plementation
    ... // compareOrder, iterateAxis, getAttributeNode, ...
}

public class ElementNode extends AbstractElementNode { // PROBLEM: Only child of AbstractElement
  Node
    protected AttributeNode createAttributeNode() { ... }
    ... // createChildren, hasChildNodes, getLocalPart, ...
}
```

Why this is a problem:

- Each abstract class has exactly one subclass, so the abstraction never gives any benefit. It only splits one idea over two files that have to be read together.

- The only abstract method (`createAttributeNode`) has just one implementation, so the abstract class is unnecessary.

c. Steps to refactor and justification:

- **Step 1:** Move every field and method of `AbstractRootNode` into `RootNode` and move the `super(...)` call into the `RootNode` constructor. `RootNode` now extends `AbstractNode` directly, and `AbstractRootNode.java` is deleted.
- **Step 2:** Do the same for `AbstractElementNode` and `ElementNode`. The abstract `createAttributeNode()` becomes a `private` method because nothing overrides it any more, and `AbstractElementNode.java` is deleted.
- **Step 3:** Search the project to confirm that nothing else used the two deleted classes. The code was only moved, not changed, so behaviour stays the same.

d. Our implementation:

```
public class RootNode extends AbstractNode {
    public RootNode(DetailAST detailAst) {
        super(new GenericTreeInfo(Configuration.newConfiguration()));
        this.detailAst = detailAst;
    }
    ... // all former AbstractRootNode members are now here
}

public class ElementNode extends AbstractNode {
    public ElementNode(AbstractNode root, AbstractNode parent, DetailAST detailAst,
            int depth, int indexAmongSiblings) {
        super(root.getTreeInfo());
        ...
        this.detailAst = detailAst;
    }
    private AttributeNode createAttributeNode() { ... }
    ... // all former AbstractElementNode members are now here
}
```

Why this solves the issue:

- The two abstract classes no longer exist, so there is no class with a single child left in this package. There are two fewer classes and two fewer files to maintain.
- Each node type is now fully described in one class. CheckStyle still compiles and runs.

Instance 2: LongMethodClass in ImportControlLoader.startElement

a. Reason for choosing the Code Smell instance:

- **Procedural Bloat in SAX Parsing:** The `startElement` callback in `ImportControlLoader` handles multiple distinct XML tags in a single procedural monolith, combining configuration attribute extraction with parse-tree stack operations.
- **High Readability & Testability Payoff:** Extracting the tag processing into dedicated private helper methods turns `startElement` into a clean, readable dispatcher while preserving all parsing behavior.

b. Location within the CheckStyle code base [original code]:

`com.puppycrawl.tools.checkstyle.checks.imports.ImportControlLoader`, method `startElement`:

```
    public void startElement(String namespaceUri, String localName,
                             String qualifiedName, Attributes attributes)
            throws SAXException {
        if ("import-control".equals(qualifiedName)) { // PROBLEM: First branch mixing tag parsin
  g and stack management
            final String pkg = safeGet(attributes, PKG_ATTRIBUTE_NAME);
            final MismatchStrategy strategyOnMismatch = getStrategyForImportControl(attributes);
            final boolean regex = containsRegexAttribute(attributes);
            stack.push(new PkgImportControl(pkg, regex, strategyOnMismatch));
        }
        else if (SUBPACKAGE_ELEMENT_NAME.equals(qualifiedName)) { // PROBLEM: Second branch with
   9 lines of local logic
            ... // read attributes, create PkgImportControl, add child, push
        }
        else if (FILE_ELEMENT_NAME.equals(qualifiedName)) { // PROBLEM: Third branch with 8 line
  s of local logic
            ... // read attributes, create FileImportControl, add child, push
        }
        else {
            final AbstractImportRule rule = createImportRule(qualifiedName, attributes);
            stack.peek().addImportRule(rule);
        }
    }
```

Why this is a problem:

- One method does four different jobs, with many local variables and branches. To change one element type you have to read and understand all of them.
- The method is hard to test, because each case is only reachable through the large `if` / `else` chain.

c. Steps to refactor and justification:

- **Step 1:** Extract each of the first three branches into its own `private` method: `startImportControlElement`, `startSubpackageElement` and `startFileElement`. The code inside is copied unchanged.
- **Step 2:** Leave `startElement` as a short dispatcher that keeps the same conditions in the same order and calls the right method, so behaviour is identical.

d. Our implementation:

```
    public void startElement(String namespaceUri, String localName,
                             String qualifiedName, Attributes attributes)
            throws SAXException {
        if ("import-control".equals(qualifiedName)) {
            startImportControlElement(attributes);
        }
        else if (SUBPACKAGE_ELEMENT_NAME.equals(qualifiedName)) {
            startSubpackageElement(attributes);
        }
        else if (FILE_ELEMENT_NAME.equals(qualifiedName)) {
            startFileElement(attributes);
        }
        else {
            final AbstractImportRule rule = createImportRule(qualifiedName, attributes);
            stack.peek().addImportRule(rule);
        }
    }

    private void startImportControlElement(Attributes attributes) throws SAXException { ... }
    private void startSubpackageElement(Attributes attributes) throws SAXException { ... }
    private void startFileElement(Attributes attributes) throws SAXException { ... }
```

Why this solves the issue:

- `startElement` is now about 15 lines and only decides which handler to call. Each handler is small, has one job and a name that says what it does.
- A change to one element type now touches only one small method. CheckStyle still compiles and runs.